Feedforward Neural Networks (Multilayer Perceptrons)

George Michailidis

gmichail@ucla.edu

STAT 426

Building and Training a Predictive Model

To build a predictive model $y = f(x) + \text{error}$ within the Statistical Learning Theory framework, we require:

- Data: A training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ drawn from spaces $\mathcal{X} \times \mathcal{Y}$.
- Model: A hypothesis class $\mathcal{H}$ (the set of all possible functions f of interest); i.e., the statistical model.
- Criterion for Success: A loss function $L(y, f(x))$ to measure how well the model f predicts labels y.

Using the Empirical Risk Minimization framework, we find the "best" $f \in \mathcal{H}$ by minimizing average loss:

$$\hat{f} = \underset{f \in \mathcal{H}}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} L(f(x_i), y_i)$$

Recap: Linear Regression - I

Data:

- Input: Feature vector $\mathbf{x} \in \mathbb{R}^p$
- Outcome: Continuous scalar $y \in \mathbb{R}$

Model Parameterization

$$f(\mathbf{x}) = \beta_0 + \sum_{j=1}^{p} \beta_j x_j = \boldsymbol{\beta}^\top \mathbf{x}$$

Key Assumption: The relationship is **globally linear** and **additive**.

3. Loss Function: Squared Error Loss

$$L(y, f(\mathbf{x})) = (y - f(\mathbf{x}))^2 = (y - \boldsymbol{\beta}^\top \mathbf{x})^2$$

Recap: Linear Regression - II

Based on the selected loss function and the empirical risk minimization framework: Optimization Objective (Least Squares Criterion) We search for the parameters $\hat{\boldsymbol{\beta}}$ that minimize $L(\cdot,\cdot)$; i.e., the Mean Squared Error:

$$\begin{aligned}\hat{\boldsymbol{\beta}} &= \underset{\boldsymbol{\beta}\in\mathbb{R}^p}{\operatorname{argmin}} \frac{1}{n}\sum_{i=1}^{n}(y_i - \boldsymbol{\beta}^\top \mathbf{x}_i)^2 \\ &= \underset{\boldsymbol{\beta}\in\mathbb{R}^p}{\operatorname{argmin}} \frac{1}{n}\|\mathbf{y} - \mathbf{X}\boldsymbol{\beta}\|_2^2\end{aligned}$$

Analytic Solution (Normal Equations) Taking the gradient $\nabla_{\boldsymbol{\beta}}$ and setting it to zero yields the closed-form solution:

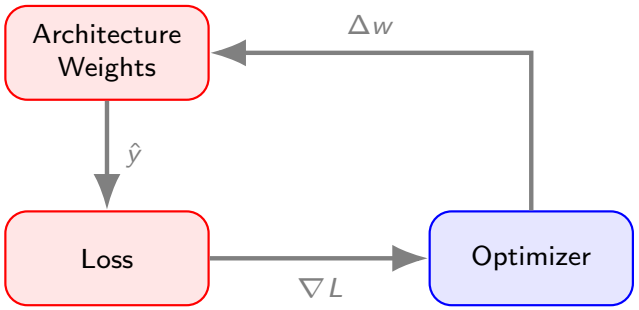
$$\hat{\boldsymbol{\beta}} = (\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$$

Feedforward Neural Networks: Key Components

1. Architecture
   How data flows from input to output (Layers, Weights, Biases).

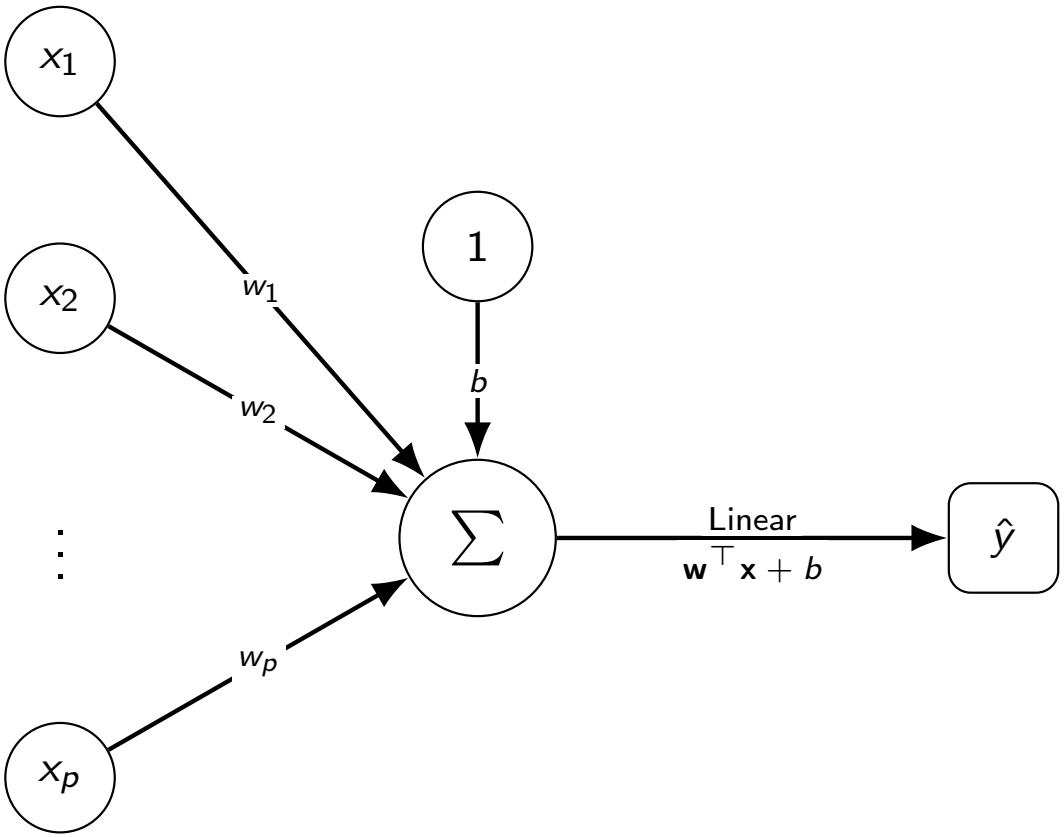
2. Loss Function

3. Optimization
   The algorithm that updates parameters to minimize loss (e.g., Minibatch Stochastic Gradient Descent).



Linear Regression from a Neural Network Perspective - I

Architecture



Linear Regression from a Neural Network Perspective - II

1. Model Architecture

A single fully connected layer with identity activation:

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b$$

Input $\mathbf{x} \in \mathbb{R}^p \rightarrow$ *Single Output Neuron* $\rightarrow$ *Identity Function* $\rightarrow$ *Output* $\hat{y}$

2. Loss Function (MSE)

We minimize the average squared error over n training examples:

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^{n} \frac{1}{2} \left(\underbrace{\mathbf{w}^\top \mathbf{x}_i + b}_{\hat{y}_i} - y_i \right)^2$$

3. Optimization Goal

Find the parameters $\hat{\mathbf{w}}, \hat{b}$ that minimize the loss on training data set $\mathcal{D}$:

$$\hat{\mathbf{w}}, \hat{b} = \underset{\mathbf{w}, b}{\operatorname{argmin}}\, L(\mathbf{w}, b)$$

Remarks: The Neural Shift

- **Structural Equivalence:**
  Linear regression corresponds exactly to a linear neural network consisting of a single fully connected layer and an identity activation function.

- **Notation Translation:**
  We switch from classical statistical notation to Deep Learning notation:

$$\underbrace{\beta_0}_{\text{Intercept}} \longrightarrow \underbrace{b}_{\text{Bias}} \quad \text{and} \quad \underbrace{\boldsymbol{\beta}}_{\text{Coefficients}} \longrightarrow \underbrace{\mathbf{w}}_{\text{Weights}}$$

- **Algorithmic Shift:**
  Instead of the closed-form analytic solution (Normal Equations / Least Squares), we use iterative optimization (Stochastic Gradient Descent).
  - *Reason:* Iterative methods scale better to massive datasets where matrix inversion is computationally prohibitive.

Recap: Logistic Regression Model - I

The logistic model (logit) is a statistical model that predicts the log-odds of an event as a linear combination of p predictor variables.

Key Task: Predict a binary categorical label $y \in \{0, 1\}$ given input features $\mathbf{x} \in \mathbb{R}^p$.

- $y_i = 1$: Event occurs (e.g., Image is a Cat)
- $y_i = 0$: Event does not occur (e.g., Image is a Dog)

The Probability Model The probability π_i that the i-th event occurs is given by the sigmoid function:

$$\pi_i(\mathbf{x}_i, \boldsymbol{\beta}) = \mathbb{P}(y_i = 1 \mid \mathbf{x}_i, \boldsymbol{\beta}) = \frac{1}{1 + \exp(-\mathbf{x}_i^\top \boldsymbol{\beta})}$$

Recap: Logistic Regression Model - II

1. Likelihood Function

Assuming independent samples, the likelihood is the product of the probabilities:

$$L(\boldsymbol{\beta}) = \prod_{i=1}^{m} \left[\pi_i(\mathbf{x}_i, \boldsymbol{\beta})\right]^{y_i} \left[1 - \pi_i(\mathbf{x}_i, \boldsymbol{\beta})\right]^{1-y_i}$$

2. Log-Likelihood Function

Taking the log turns the product into a sum:

$$\ell(\boldsymbol{\beta}) = \sum_{i=1}^{m} \Big[y_i \log \pi_i(\mathbf{x}_i, \boldsymbol{\beta}) + (1 - y_i) \log\big(1 - \pi_i(\mathbf{x}_i, \boldsymbol{\beta})\big) \Big]$$

3. Simplified Form

Substituting the sigmoid formula and simplifying yields:

$$\begin{aligned} \ell(\boldsymbol{\beta}) &= \sum_{i=1}^{m} \left[y_i(\mathbf{x}_i^\top \boldsymbol{\beta}) + \log\left(\frac{1}{1 + \exp(\mathbf{x}_i^\top \boldsymbol{\beta})} \right) \right] \\ &= \sum_{i=1}^{m} \Big[y_i(\mathbf{x}_i^\top \boldsymbol{\beta}) - \log\big(1 + \exp(\mathbf{x}_i^\top \boldsymbol{\beta})\big) \Big] \end{aligned}$$

Mapping Logistic Regression to our General Framework - I

Data:

- Input: Feature vector $\mathbf{x} \in \mathbb{R}^p$
- Outcome: Binary scalar $y \in \{0, 1\}$ (Bernoulli)

Model Parameterization

$$\pi(\mathbf{x}) = P(y = 1|\mathbf{x}) = \frac{1}{1 + e^{-\boldsymbol{\beta}^\top \mathbf{x}}} = \sigma(\boldsymbol{\beta}^\top \mathbf{x})$$

Key Assumption: The **log-odds** are linear: $\log(\frac{\pi}{1-\pi}) = \boldsymbol{\beta}^\top \mathbf{x}$.

3. Loss Function: Negative Log-Likelihood (Cross-Entropy)

$$L(y, \pi(\mathbf{x})) = -\left[y \log \pi(\mathbf{x}) + (1 - y) \log(1 - \pi(\mathbf{x}))\right]$$

Mapping Logistic Regression to our General Framework - II

Based on the maximum likelihood principle (equivalently, minimizing NLL):
Optimization Objective We search for the parameters $\hat{\boldsymbol{\beta}}$ that minimize the total loss:

$$\begin{aligned}\hat{\boldsymbol{\beta}} &= \underset{\boldsymbol{\beta}\in\mathbb{R}^p}{\operatorname{argmin}} \sum_{i=1}^{n} L(y_i, \pi(\mathbf{x}_i)) \\ &= \underset{\boldsymbol{\beta}\in\mathbb{R}^p}{\operatorname{argmin}} \sum_{i=1}^{n} -\left[y_i \log \pi_i + (1-y_i)\log(1-\pi_i)\right]\end{aligned}$$

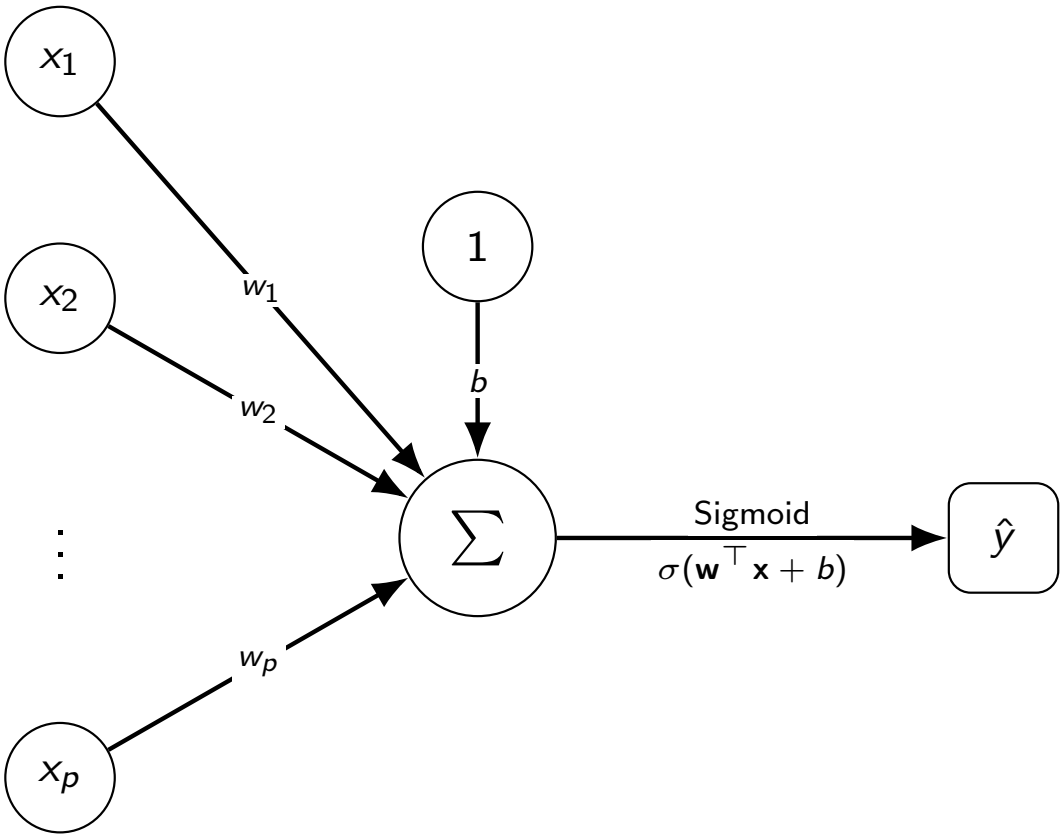
Solution (No Closed Form)
Unlike linear regression, taking the gradient $\nabla_{\boldsymbol{\beta}}$ and setting it to zero does not yield a closed-form algebraic solution.

$$\nabla_{\boldsymbol{\beta}}\ell(\boldsymbol{\beta}) = \mathbf{X}^\top(\mathbf{y}-\boldsymbol{\pi}) = 0 \quad \text{(Transcendental Equation)}$$

We must use iterative methods such as Newton-Raphson or Gradient Descent.

Logistic Regression from a Neural Network Perspective - I

Architecture



Logistic Regression from a Neural Network Perspective - II

1. Model Architecture

A single fully connected layer with sigmoid activation:

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}}$$

Input $\mathbf{x} \in \mathbb{R}^p \rightarrow$ *Single Output Neuron* $\rightarrow$ *Sigmoid Function* $\rightarrow$ *Probability* $\hat{y}$

2. Loss Function (Binary Cross-Entropy)

We minimize the negative log-likelihood over n training examples:

$$L(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^{n} [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$$

3. Optimization Goal

Find the parameters $\hat{\mathbf{w}}, \hat{b}$ that minimize the loss on training data set $\mathcal{D}$:

$$\hat{\mathbf{w}}, \hat{b} = \underset{\mathbf{w}, b}{\operatorname{argmin}}\, L(\mathbf{w}, b)$$

Remarks: The Neural Shift

- **Structural Equivalence:**
  Logistic regression corresponds exactly to a linear neural network consisting of a single fully connected layer with a sigmoid activation function.

- **Notation Translation:**
  We switch from classical statistical notation to Deep Learning notation:

$$\underbrace{\beta_0}_{\text{Intercept}} \longrightarrow \underbrace{b}_{\text{Bias}} \quad \text{and} \quad \underbrace{\boldsymbol{\beta}}_{\text{Coefficients}} \longrightarrow \underbrace{\mathbf{w}}_{\text{Weights}}$$

- **Algorithmic Shift:**
  Unlike linear regression, there is no closed-form solution. We must use iterative optimization (Stochastic Gradient Descent).
  - *Reason:* The non-linearity of the sigmoid function makes analytic inversion impossible; gradient-based learning is required.

Recap: Multi-class Classification Model - I

The multinomial logistic model (Softmax regression) generalizes logistic regression to predict probabilities for $K > 2$ mutually exclusive classes.

Key Task: Predict a categorical label $y \in \{1, \ldots, K\}$ given input features $\mathbf{x} \in \mathbb{R}^p$.

- $y = 1$ (Airplane), $y = 2$ (Car), $\ldots$, $y = 10$ (Ship)

The Probability Model (Softmax) The probability that the observation belongs to class k is given by the Softmax function:

$$\pi_k(\mathbf{x}, \mathbf{B}) = P(y = k \mid \mathbf{x}) = \frac{\exp(\boldsymbol{\beta}_k^\top \mathbf{x})}{\sum_{j=1}^{K} \exp(\boldsymbol{\beta}_j^\top \mathbf{x})}$$

where $\boldsymbol{\beta}_k$ is the weight vector specific to class k and $\mathbf{B}$ a $p \times k$ matrix containing the β_k's.

Recap: Multi-class Classification Model - II

1. Likelihood Function

Using one-hot encoding where $y_{ik} = 1$ if observation i is class k (else 0), the likelihood is the product over all n samples and K classes:

$$L(\mathbf{B}) = \prod_{i=1}^{n} \prod_{k=1}^{K} \left[\pi_k(\mathbf{x}_i)\right]^{y_{ik}}$$

2. Log-Likelihood Function

Taking the log turns the products into sums:

$$\ell(\mathbf{B}) = \sum_{i=1}^{n} \sum_{k=1}^{K} y_{ik} \log \pi_k(\mathbf{x}_i)$$

3. Simplified Form

Substituting the Softmax probability:

$$\ell(\mathbf{B}) = \sum_{i=1}^{n} \sum_{k=1}^{K} y_{ik} \log \left(\frac{e^{\boldsymbol{\beta}_k^\top \mathbf{x}_i}}{\sum_j e^{\boldsymbol{\beta}_j^\top \mathbf{x}_i}} \right) = \sum_{i=1}^{n} \left[\sum_{k=1}^{K} y_{ik} (\boldsymbol{\beta}_k^\top \mathbf{x}_i) - \log \left(\sum_{j=1}^{K} e^{\boldsymbol{\beta}_j^\top \mathbf{x}_i} \right) \right]$$

Mapping Multi-class to our General Framework - I

Data:

- Input: Feature vector $\mathbf{x} \in \mathbb{R}^p$
- Outcome: Categorical $y \in \{1, \ldots, K\}$ (One-hot encoded $\mathbf{y} \in \{0, 1\}^K$)

Model Parameterization

$$\hat{\mathbf{y}} = \text{Softmax}(\mathbf{B}^\top \mathbf{x} + \mathbf{b}) \implies \hat{y}_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

Key Assumption: The **logits** (z_k) are linear: $z_k = \boldsymbol{\beta}_k^\top \mathbf{x}$. The output is a probability distribution over classes.

3. Loss Function: Categorical Cross-Entropy

$$L(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$

Digression: One-Hot Encoding for Multiclass Labels

Setting: Multiclass classification with K classes.

$$y \in \{0,1\}^K$$

Definition: The label y is a vector of length K with

- exactly one entry equal to 1,
- all other entries equal to 0.

Example ($K = 4$):

$$\mathcal{Y} = \{\text{cat},\ \text{dog},\ \text{bird},\ \text{fish}\}$$

Class	One-hot encoded y
cat	$(1,\ 0,\ 0,\ 0)^\top$
dog	$(0,\ 1,\ 0,\ 0)^\top$
bird	$(0,\ 0,\ 1,\ 0)^\top$
fish	$(0,\ 0,\ 0,\ 1)^\top$

Compact notation: If the true class is k, then

$$y = e_k,$$

the k-th standard basis vector in $\mathbb{R}^K$.

Mapping Multi-class to our General Framework - II

Based on the maximum likelihood principle (minimizing Negative Log-Likelihood):

Optimization Objective

We search for the matrix of parameters $\hat{\mathbf{W}}$ that minimizes the total loss:

$$\hat{\mathbf{B}} = \underset{\mathbf{B}}{\operatorname{argmin}} \sum_{i=1}^{n} L(\mathbf{y}_i, \hat{\mathbf{y}}_i) = \underset{\mathbf{W}}{\operatorname{argmin}} \left(-\sum_{i=1}^{n} \sum_{k=1}^{K} y_{ik} \log(\hat{y}_{ik}) \right)$$

Solution (No Closed Form)

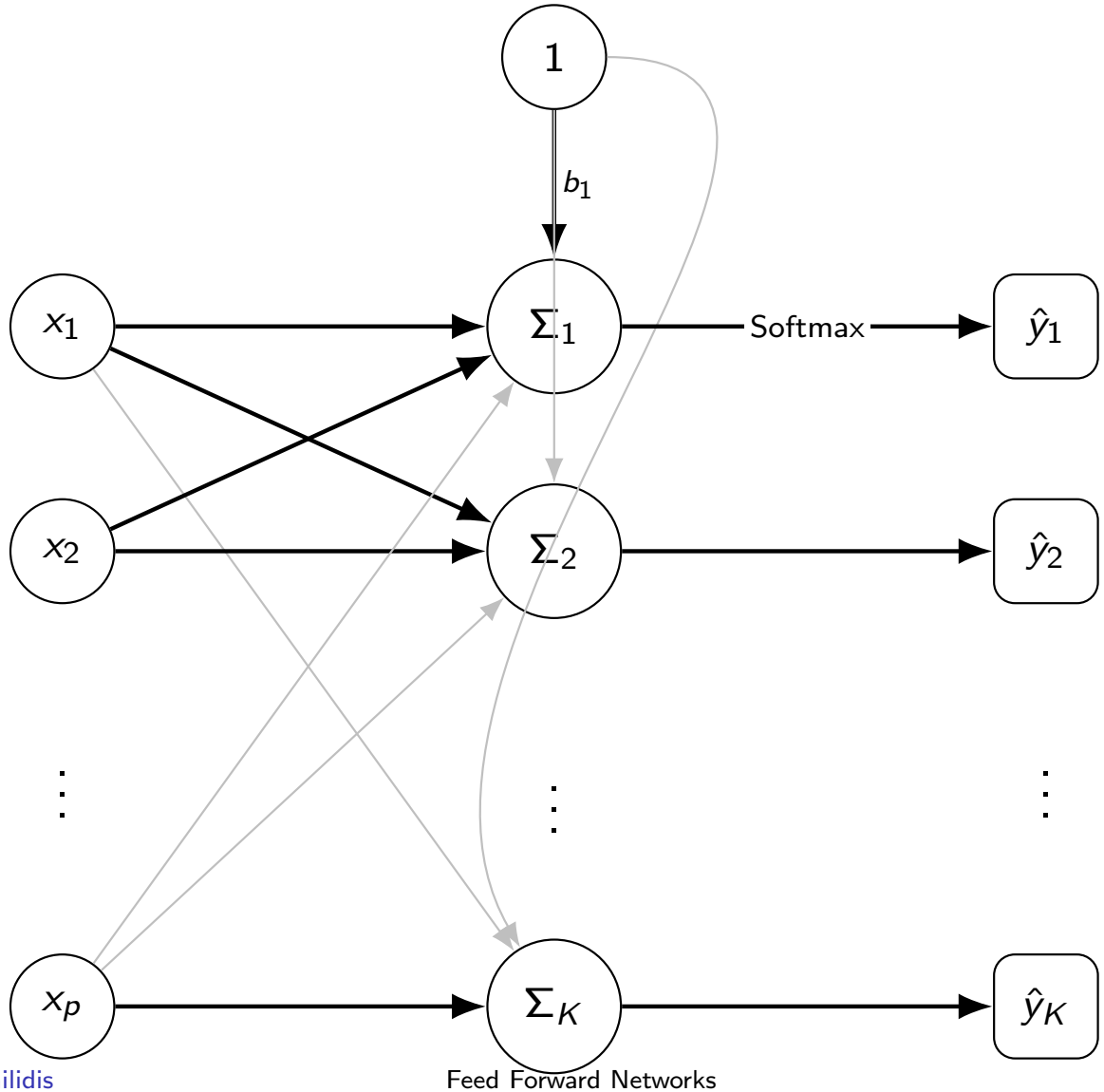
Similar to binary logistic regression, taking the gradient $\nabla_{\mathbf{B}}$ and setting it to zero does not yield a closed-form algebraic solution due to the Softmax non-linearity.

$$\nabla_{\beta_k} \ell(\mathbf{B}) = \sum_{i=1}^{n} (\hat{y}_{ik} - y_{ik}) \mathbf{x}_i = 0$$

We must use iterative methods such as Newton-Raphson or Gradient Descent.

Multi-class Model from a Neural Network Perspective - I

Architecture



Multi-class Model from a Neural Network Perspective - II

1. Model Architecture

A single fully connected layer with K output neurons and Softmax activation:

$$\hat{\mathbf{y}} = \text{Softmax}(\mathbf{W}^\top \mathbf{x} + \mathbf{b})$$

Input $\mathbf{x} \in \mathbb{R}^p \rightarrow K$ *Output Neurons* $\rightarrow$ *Softmax* $\rightarrow$ *Probability Dist.* $\hat{\mathbf{y}}$

2. Loss Function (Categorical Cross-Entropy)

We minimize the negative log-likelihood over n training examples:

$$L(\mathbf{W}, \mathbf{b}) = -\frac{1}{n} \sum_{i=1}^{n} \sum_{k=1}^{K} y_{ik} \log(\hat{y}_{ik})$$

3. Optimization Goal

Find the matrix $\hat{\mathbf{W}}$ and bias vector $\hat{\mathbf{b}}$ that minimize loss on $\mathcal{D}$:

$$\hat{\mathbf{W}}, \hat{\mathbf{b}} = \underset{\mathbf{W}, \mathbf{b}}{\text{argmin}}\, L(\mathbf{W}, \mathbf{b})$$

Remarks: The Neural Shift

- **Structural Equivalence:**
  Multinomial Logistic regression corresponds exactly to a linear neural network with K output neurons followed by a Softmax activation function.

- **Notation Translation:**
  We switch from classical statistical notation to Deep Learning notation:

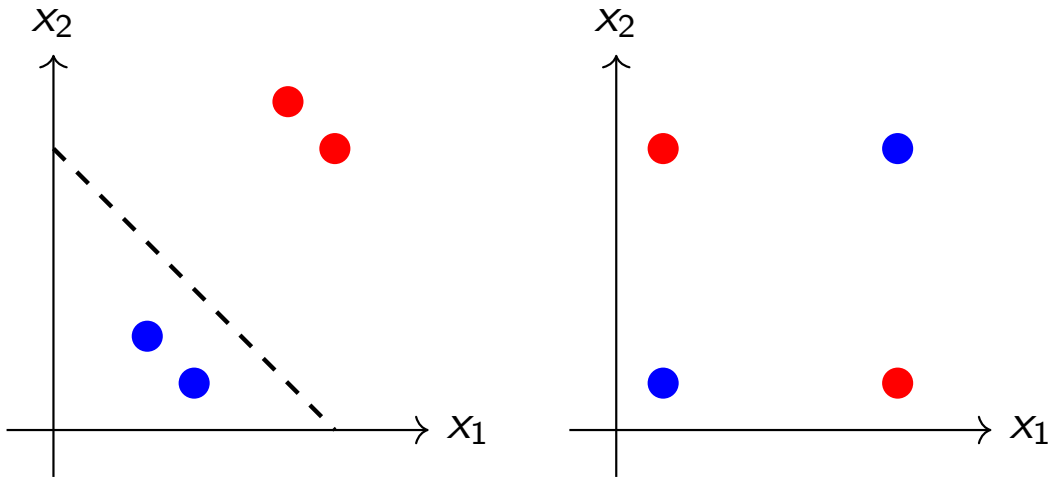
$$\underbrace{\boldsymbol{\beta}_k}_{\text{Class Coefficients}} \longrightarrow \underbrace{\mathbf{W}_{:,k}}_{\text{Class Weights}} \qquad (\mathbf{W} \text{ becomes a matrix})$$

- **Algorithmic Shift:**
  Just like binary logistic regression, there is no closed-form solution. We must use iterative optimization (Gradient/Stochastic Gradient Descent).
  - *Reason:* The Softmax normalization term couples all classes together, making analytic inversion impossible.

The Limitation of Linear Models

The Problem: Single-layer networks (Linear/Logistic/Softmax) are limited in their expressiveness.

- They imply a monotonic relationship: increasing x_i must always increase (or always decrease) the output.
- For example, they cannot solve the **XOR Problem** or handle complex interactions between features.



Linear (Solvable) XOR (Impossible Linearly)

To model complex data, we need Hidden Layers and Non-linearities.

Incorporating Hidden Layers

To overcome linearity, we introduce one or more **Hidden Layers** between the input and the output.

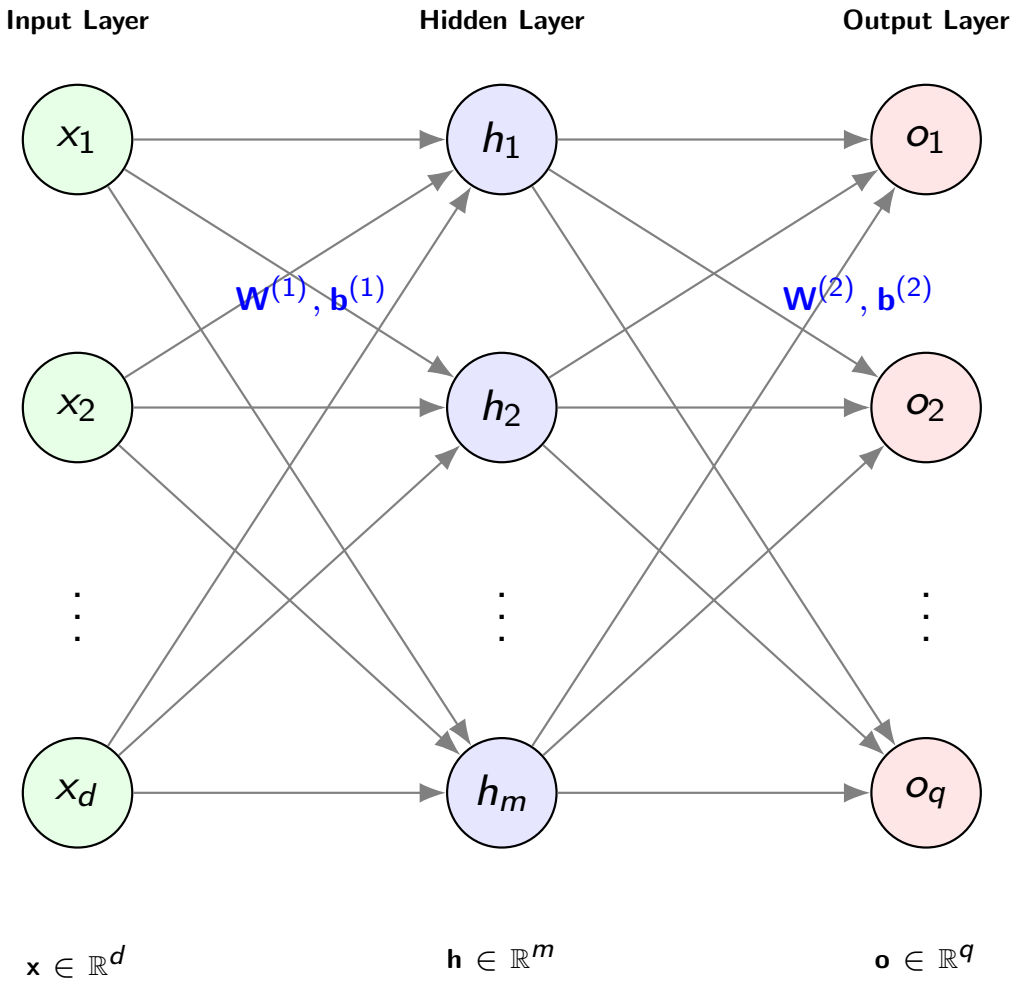
Architecture Change:

1. Input Layer: $\mathbf{x} \in \mathbb{R}^d$
2. **Hidden Layer**: $\mathbf{h} \in \mathbb{R}^h$
   - Fully connected to inputs.
   - Output of this layer becomes input to the next.
3. Output Layer: $\mathbf{o} \in \mathbb{R}^q$

This architecture is called a Multilayer Perceptron (MLP).

Note: "Perceptron" is historical terminology. Modern MLPs use smooth activation functions, not threshold steps.

MLP Architecture Visualization



Mathematical Formulation

Consider an MLP with single hidden layer with h hidden units hence, depth=1, width=h.

Step 1: The Hidden Layer Calculation

$$\mathbf{h} = \sigma(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$$

Step 2: The Output Layer Calculation

$$\mathbf{o} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

Dimensions:

- Input $\mathbf{x} \in \mathbb{R}^d$, Batch size $n \rightarrow \mathbf{X} \in \mathbb{R}^{n \times d}$
- Weights $\mathbf{W}^{(1)} \in \mathbb{R}^{d \times h}$, Bias $\mathbf{b}^{(1)} \in \mathbb{R}^{1 \times h}$
- Weights $\mathbf{W}^{(2)} \in \mathbb{R}^{h \times q}$, Bias $\mathbf{b}^{(2)} \in \mathbb{R}^{1 \times q}$

Crucial Element: $\sigma(\cdot)$ is a non-linear activation function.

Why do we need Activation Functions?

Why not just stack linear layers?

$$\mathbf{h} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}$$

$$\mathbf{o} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

Substituting **h** into **o**:

$$\begin{aligned}\mathbf{o} &= \mathbf{W}^{(2)}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}) + \mathbf{b}^{(2)} \\ &= \underbrace{(\mathbf{W}^{(2)}\mathbf{W}^{(1)})}_{\mathbf{W}_{new}}\mathbf{x} + \underbrace{(\mathbf{W}^{(2)}\mathbf{b}^{(1)} + \mathbf{b}^{(2)})}_{\mathbf{b}_{new}}\end{aligned}$$

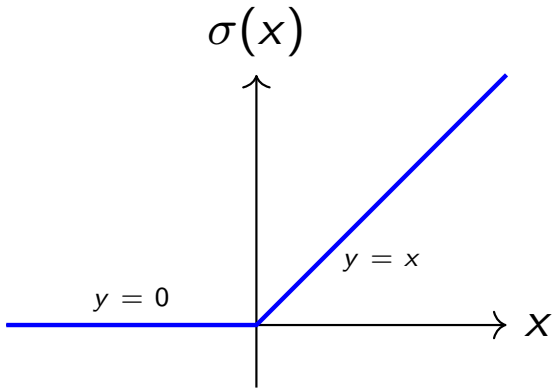
- An affine function of an affine function is still an affine function.
- Without non-linearity σ, the hidden layer is mathematically useless (it collapses into a single linear model).

Activation Functions: ReLU

The most popular choice in modern Deep Learning:
Rectified Linear Unit (ReLU)

$$\mathrm{ReLU}(x) = \max(x, 0)$$

- **Piecewise linear**: Preserves many optimization properties of linear models.
- **Computation**: Extremely fast (simple thresholding).
- **Gradient**: 1 if $x > 0$, 0 if $x < 0$. Avoids the "vanishing gradient" problem for positive inputs.

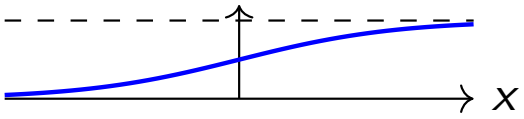


Activation Functions: Sigmoid and Tanh

Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

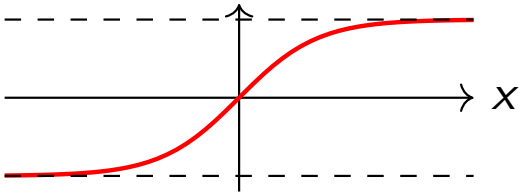
- Range: $(0, 1)$
- Used in: Binary output layers, LSTM gates.
- Issue: Vanishing gradients (derivative near 0 for large $|x|$).



Tanh (Hyperbolic Tangent)

$$\tanh(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

- Range: $(-1, 1)$
- Centered at 0 (often converges faster than sigmoid).
- Issue: Still suffers from vanishing gradients.



General Notation for Deep Networks

We can stack multiple hidden layers (hence the terminology deep NN). Let L be the number of layers.

- $\mathbf{h}^{(0)} = \mathbf{x}$ (Input)
- $\mathbf{h}^{(l)}$ is the output of hidden layer l.

Recursive definition:

$$\mathbf{h}^{(l)} = \sigma_l(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)})$$

for $l = 1, \ldots, L$.

- $\mathbf{W}^{(l)}$: Weights connecting layer $l-1$ to layer l.
- $\mathbf{b}^{(l)}$: Bias for layer l.
- σ_l: Activation function (usually ReLU for hidden, specific function for output).

Universal Approximation Theorem - I

Why do MLPs work?

The **Universal Approximation Theorem** states that a feedforward network can approximate any continuous function $f : \mathbb{R}^n \to \mathbb{R}^m$ to arbitrary precision.

Universal Approximation Theorem - II

Statement (Informal): A feedforward neural network with

- a **single hidden layer**,
- a **finite number of neurons**, and
- a **nonlinear, non-polynomial activation function** (e.g., sigmoid, `tanh`, ReLU),

can approximate **any continuous function**

$$f : K \subset \mathbb{R}^n \to \mathbb{R}^m$$

uniformly on compact sets K, to **arbitrary precision**.

Formally: For any $\varepsilon > 0$, there exists a network f_θ such that

$$\sup_{x \in K} \|f(x) - f_\theta(x)\| < \varepsilon.$$

Important caveats

- This is an **existence** result (not a training guarantee).
- The number of required neurons may be very large.
- Approximation does **not** imply good generalization.

Example 1: MLP for Regression

Task: Predict a continuous value $y \in \mathbb{R}$ (e.g., House Price).

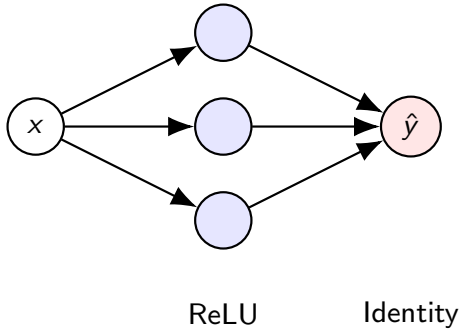
Architecture:

- Hidden Layer: ReLU activation. $\mathbf{h} = \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$
- Output Layer: Identity activation (no non-linearity).

$$\hat{y} = \mathbf{w}^{(2)\top}\mathbf{h} + b^{(2)}$$

Loss Function: Mean Squared Error (MSE).

$$L(\mathbf{y}, \hat{\mathbf{y}}) = \frac{1}{2}\|\mathbf{y} - \hat{\mathbf{y}}\|^2$$



Example 2: MLP for Binary Classification

Task: Predict a class $y \in \{0, 1\}$ (e.g., Spam vs Not Spam).

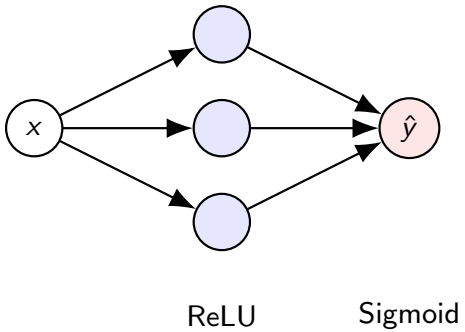
Architecture:

- Hidden Layer: ReLU activation. $\mathbf{h} = \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$
- Output Layer: Sigmoid activation.

$$\hat{y} = \sigma(\mathbf{w}^{(2)\top}\mathbf{h} + b^{(2)}) = \frac{1}{1 + e^{-o}}$$

Loss Function: Binary Cross-Entropy.

$$L(y, \hat{y}) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$



Example 3: MLP for Multi-class Classification

Task: Predict $y \in \{1, \ldots, K\}$ (e.g., Digit 0-9).

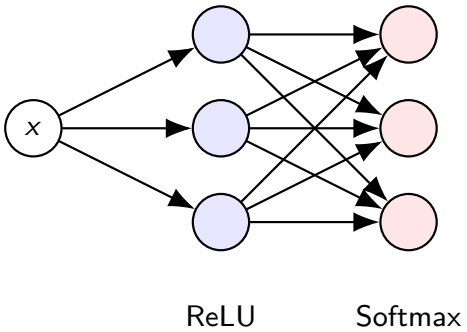
Architecture:

- Hidden Layer: ReLU activation. $\mathbf{h} = \text{ReLU}(\mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)})$
- Output Layer: Softmax activation with K nodes.

$$\hat{y}_k = \frac{\exp(o_k)}{\sum_j \exp(o_j)}, \quad \text{where } \mathbf{o} = \mathbf{W}^{(2)}\mathbf{h} + \mathbf{b}^{(2)}$$

Loss Function: Categorical Cross-Entropy.

$$L(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$



Summary: The MLP Framework

1. Linearity is not enough: To learn complex patterns, we need hidden layers.
2. **Non-linearity is Key:** We must insert an activation function (ReLU, Sigmoid, Tanh) after every hidden affine transformation, or the model collapses back to a linear model.
3. Architecture defines Task:
   - **Input:** Matches data dimension (d).
   - **Hidden:** Hyperparameter (width and depth).
   - **Output:** Matches target (1 for regression/binary, K for multi-class).
4. Optimization: We still use (Stochastic) Gradient Descent, but gradients are calculated via Backpropagation (Chain Rule).

The Training Pipeline: Forward and Backward

Training a Neural Network involves a cyclical process called the **Forward-Backward Loop**.

1. **Forward Propagation**:
   - Input data flows through the network layer by layer.
   - Produces the prediction $\hat{\mathbf{y}}$ and the scalar loss L.
   - **Crucial Step**: We must *cache* (store) intermediate values (activations) for later use.
2. **Backward Propagation (Backprop)**:
   - Gradients flow in the *reverse* direction (from Loss to Input).
   - Applies the **Chain Rule** of calculus recursively.
   - Computes the gradient of the loss with respect to every parameter: $\nabla_{\mathbf{W}} L$.
3. **Parameter Update**:
4. Adjust weights using the computed gradients (e.g., SGD).

Step 1: Forward Propagation (The Calculation)

Let $\mathbf{h}^{(0)} = \mathbf{x}$. For each layer $l = 1, \ldots, L$:

1. **Affine Transformation (Pre-activation):**

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}$$

2. **Non-linear Activation:**

$$\mathbf{h}^{(l)} = \sigma_l(\mathbf{z}^{(l)})$$

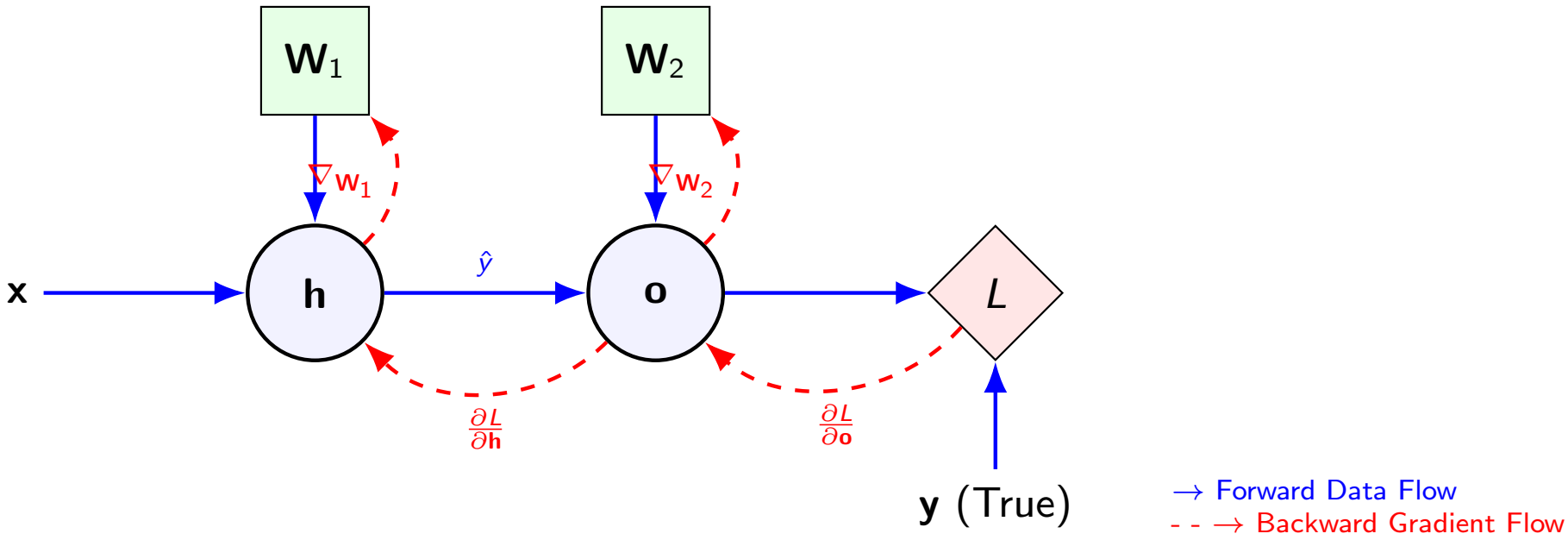
3. **Compute Loss (at the end):**

$$J = L(\mathbf{h}^{(L)}, \mathbf{y}) + \lambda\Omega(\mathbf{W}) \quad \text{(Regularization optional)}$$

Memory Cost

During Forward Prop, we **must store** $\mathbf{z}^{(l)}$ and $\mathbf{h}^{(l-1)}$ in memory. We need them to calculate derivatives during the backward pass.

Visualizing the Computational Graph



The computational graph decouples complex dependencies into simple, local operations. Backprop propagates the error signal from the loss node back to the parameters.

Step 2: Backward Propagation (The Math)

We want to find $\frac{\partial L}{\partial \mathbf{W}^{(l)}}$. We apply the **Chain Rule** starting from the output layer L.

1. **Output Error Signal:**

$$\boldsymbol{\delta}^{(L)} = \frac{\partial L}{\partial \mathbf{z}^{(L)}} = \underbrace{\frac{\partial L}{\partial \hat{\mathbf{y}}}}_{\text{Loss Deriv.}} \cdot \underbrace{\sigma'(\mathbf{z}^{(L)})}_{\text{Activ. Deriv.}}$$

2. **Backpropagate to Hidden Layers:** For $l = L-1, \ldots, 1$:

$$\boldsymbol{\delta}^{(l)} = \underbrace{(\mathbf{W}^{(l+1)\top}\boldsymbol{\delta}^{(l+1)})}_{\text{Pull from future}} \odot \underbrace{\sigma'(\mathbf{z}^{(l)})}_{\text{Local deriv.}}$$

($\odot$ is the element-wise Hadamard product)

3. **Compute Weight Gradients:**

$$\frac{\partial L}{\partial \mathbf{W}^{(l)}} = \boldsymbol{\delta}^{(l)}(\mathbf{h}^{(l-1)})^\top$$

Computational Considerations

1. Memory Intensity
   - We cannot throw away $\mathbf{h}^{(l)}$ after forward pass.
   - GPU memory is often the bottleneck because Backprop requires storing all intermediate activations.
   - **Trade-off:** Deeper networks = Linear increase in memory.

2. Vanishing/Exploding Gradients
   - In $\boldsymbol{\delta}^{(l)}$, we multiply by $\mathbf{W}^\top$ and σ' repeatedly.
   - If $\mathbf{W}$ is small or σ' is near 0 (Sigmoid), gradients vanish $\rightarrow$ No learning.
   - **Solution:** ReLU + Batch Normalization.

Summary: MLP training is simply **Function Evaluation** (Forward) followed by **differentiation via Chain Rule** (Backward), repeated iteratively.

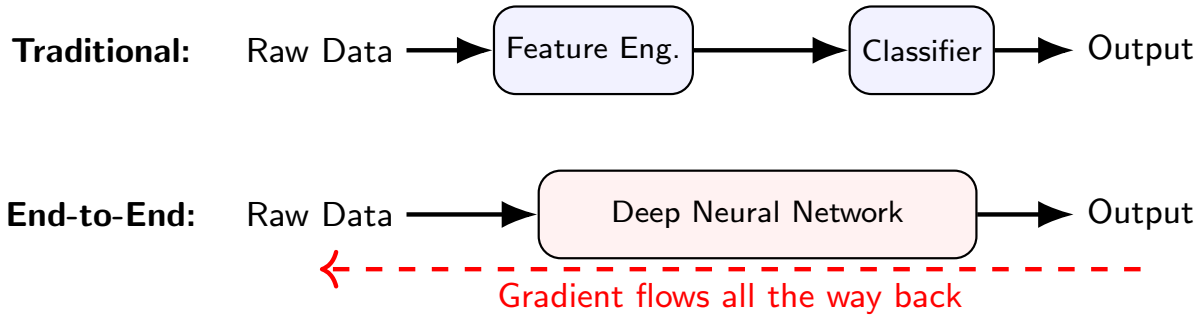
End-to-End Training: The Paradigm Shift

Traditional Machine Learning Pipeline:

- Broken into separate, manually tuned stages.
- Example (Computer Vision): Image → **Hand-crafted Features** → Classifier (SVM) → Output.
- *Limitation:* Errors accumulate; feature extractor is not optimized for the specific task.

End-to-End Deep Learning:

- The entire system is treated as a single, differentiable function.
- We map raw inputs (numerical values, pixels, audio signals, text) directly to outputs.
- Feature Learning: The network learns its own internal representations (features) specifically to minimize the final loss.



Implications of End-to-End Training

1. Global Optimization
   - Because the pipeline is fully differentiable, Backpropagation updates all parameters simultaneously.
   - The "feature extractor" (early layers) adapts to help the "classifier" (later layers), and vice versa.

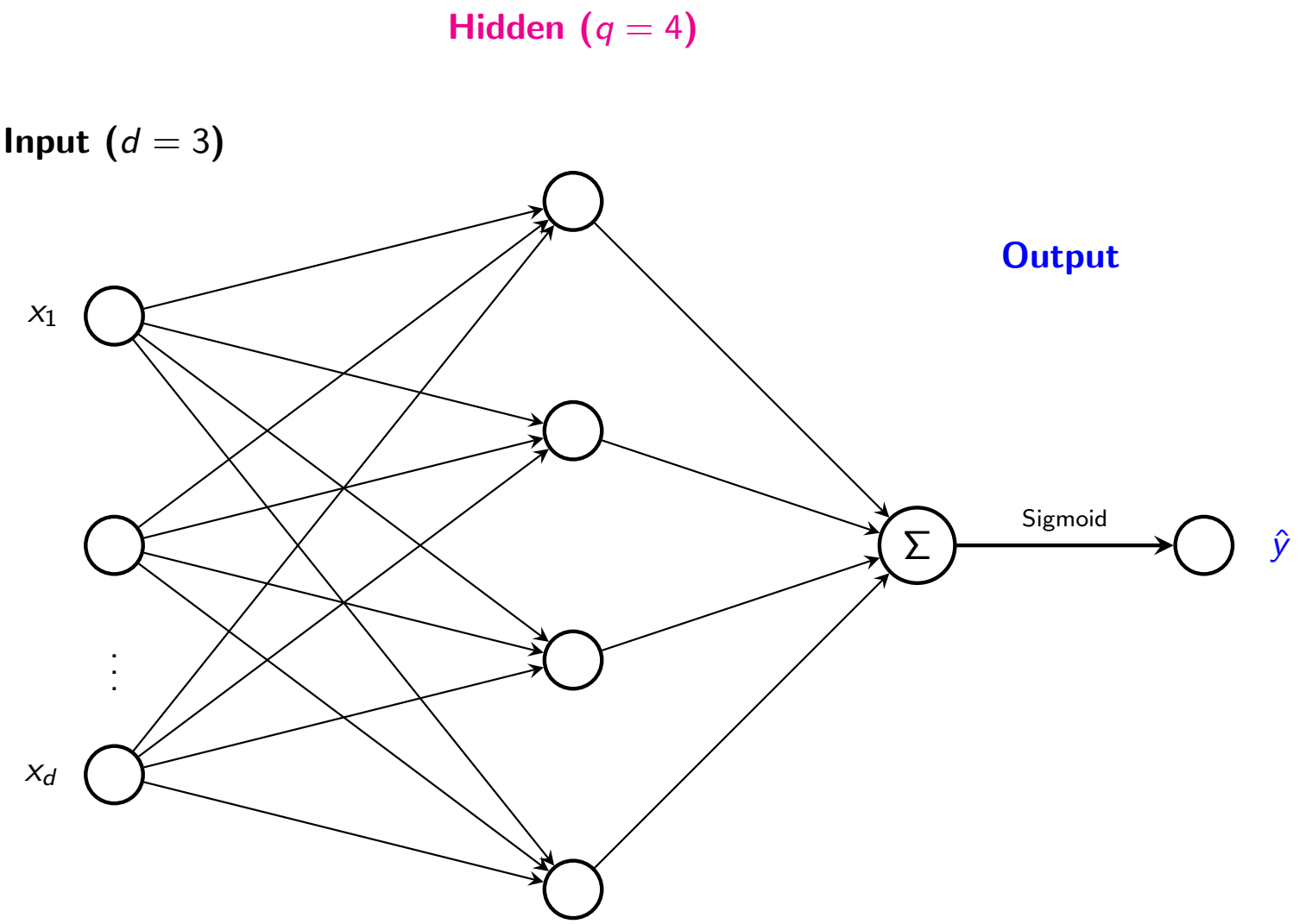
2. Data Hunger
   - Manual feature engineering encoded human prior knowledge (e.g., "edges are important").
   - Removing this prior means the model must learn these patterns from scratch.
   - Result: E2E models typically require significantly more data to generalize well.

3. Interpretability Challenges
   - Intermediate representations are learned automatically and may not be interpretable by humans (e.g., a hidden neuron might track a texture we do not have a name for).

Example: MLP for Binary Classification - Mathematical Formulation

1. **Input Layer**
   - Input: $x = [x_1, \ldots, x_d]^\top \in \mathbb{R}^d$
2. **Hidden Layer**
   - Parameters: $W_1 \in \mathbb{R}^{q \times d}, b_1 \in \mathbb{R}^q$
   - Pre-activation: $a = W_1 x + b_1$
   - Activation: $h = \mathrm{ReLU}(a) \in \mathbb{R}^q$
3. **Output Layer**
   - Parameters: $W_2 \in \mathbb{R}^{1 \times q}, b_2 \in \mathbb{R}$
   - Logit: $z = W_2 h + b_2 = \sum_{j=1}^{q} w_{2,j} h_j + b_2$
   - Pred: $\hat{y} = \sigma(z) \in (0, 1)$

Visual Illustration of the MLP



The Training Pipeline: Forward and Backward

Recall that training involves a cyclical process.

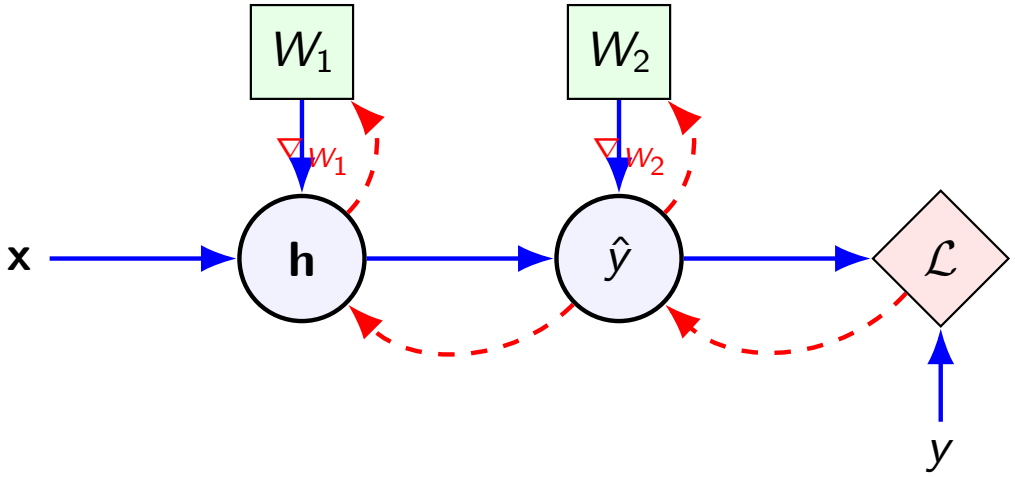
1. **Forward Propagation**:
   - Data flows Input $\rightarrow$ Output.
   - **Cache** intermediate values (a, h) for the backward pass.
2. **Backward Propagation**:
   - Error signal flows Output $\rightarrow$ Input.
   - Applies the **Chain Rule** recursively.
3. **Parameter Update**:
   - Define the model parameters: $\theta = (W_1, b_1, W_2, b_2)$.
   - Update rule (Gradient Descent):

$$\theta \leftarrow \theta - \eta \nabla_\theta L$$

   - To execute this, we need to compute the gradient with respect to each component:

$$\nabla_{W_1} L, \quad \nabla_{b_1} L, \quad \nabla_{W_2} L, \quad \nabla_{b_2} L$$

Visualizing the Computational Graph



Recap: Binary Cross-Entropy Loss

Recall that for a binary classification problem ($y \in \{0, 1\}$), we quantify the difference between the true label y and the predicted probability $\hat{y}$ using the Binary Cross-Entropy (BCE) loss.

Loss Function Definition:

$$L(y, \hat{y}) = -\big(y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})\big)$$

Visual Intuition:

- If $y = 1$: We want $\hat{y} \to 1$. The loss is $-\log(\hat{y})$. As $\hat{y} \to 1$, Loss $\to 0$.
- If $y = 0$: We want $\hat{y} \to 0$. The loss is $-\log(1 - \hat{y})$. As $\hat{y} \to 0$, Loss $\to 0$.

Goal: Minimize L with respect to the neural network parameters θ.

Step 1: Output Layer Gradients

We compute the derivative of the loss function with respect to the pre-activation z.

$$\frac{\partial L}{\partial z} = \underbrace{\frac{\partial L}{\partial \hat{y}}}_{\text{Loss Deriv.}} \cdot \underbrace{\frac{\partial \hat{y}}{\partial z}}_{\text{Sigmoid Deriv.}} = (\hat{y} - y)$$

Gradients for W_2, b_2:

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial W_2} = (\hat{y} - y) \cdot h^\top \quad \in \mathbb{R}^{1 \times q}$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial z} = (\hat{y} - y) \quad \in \mathbb{R}$$

Step 2: Propagating to Hidden Layer

We back-propagate through W_2 to the hidden nodes h.

$$\frac{\partial L}{\partial h} = \underbrace{W_2^\top}_{(q\times 1)} \cdot \underbrace{(\hat{y} - y)}_{(1\times 1)} \quad \in \mathbb{R}^{q\times 1}$$

Through the Activation (ReLU):
We element-wise multiply ($\odot$) by the derivative of ReLU (1 if $a > 0$, else 0).

$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial h} \odot \mathbf{1}_{a>0} \quad \in \mathbb{R}^{q\times 1} \tag{1}$$

Step 3: Input Layer Gradients

Finally, we compute gradients for the first layer weights W_1.

$$\frac{\partial L}{\partial W_1} = \underbrace{\frac{\partial L}{\partial a}}_{(q\times 1)} \cdot \underbrace{x^\top}_{(1\times d)} \quad \in \mathbb{R}^{q\times d}$$

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial a} \quad \in \mathbb{R}^{q\times 1}$$

Consistency Check:
Note that $\nabla_{W_1} L$ has dimensions $q \times d$, which matches the shape of the parameter matrix W_1.

Matrix Notation - Efficient Computation for s samples from $\mathcal{D}$

Forward Pass

$$
\begin{aligned}
A &= XW_1 + \mathbf{1}_s b_1^\top && \in \mathbb{R}^{s\times q} \\
H &= \mathrm{ReLU}(A) && \in \mathbb{R}^{s\times q} \\
\hat{Y} &= \sigma(HW_2 + \mathbf{1}_s b_2) && \in \mathbb{R}^{s\times 1}
\end{aligned}
$$

Backward Pass

$$
\begin{aligned}
\delta_2 &= \hat{Y} - Y && \in \mathbb{R}^{s\times 1} \\
\nabla_{W_2} L &= \frac{1}{s} H^\top \delta_2 && \in \mathbb{R}^{q\times 1} \\
\delta_1 &= (\delta_2 W_2^\top) \odot \mathbf{1}_{H>0} && \in \mathbb{R}^{s\times q} \\
\nabla_{W_1} L &= \frac{1}{s} X^\top \delta_1 && \in \mathbb{R}^{p\times q}
\end{aligned}
$$

Why is Parameter Initialization Important?

Since we use an iterative algorithm to find the best set of model parameters, proper initialization is a prerequisite for successful training. Poor choices can render the network un-trainable.

1. **Symmetry Breaking**
   - **Scenario:** If we initialize all weights to the same constant (e.g., $W = 0/c$).
   - **Consequence:** Every neuron in the hidden layer computes the same output and receives the same gradient update.
   - **Result:** The hidden units remain identical throughout training. The network effectively acts as a single linear unit, losing the power of the hidden layer dimensions.
2. **Gradient Stability (Signal Preservation)**
   - **Too Large:** Activations saturate (sigmoid/tanh) or grow uncontrollably (ReLU), leading to Exploding Gradients (NaNs/Instability).
   - **Too Small:** Signals shrink as they pass through layers. Gradients become infinitesimal during backprop, leading to Vanishing Gradients. The network stops learning.

Goal: Keep the variance of activations and gradients roughly constant across layers ($Var(y) \approx Var(h)$).

Initialization Methods

- **Weights** W_1, W_2
  - **Xavier (Glorot) Initialization** (Standard for Sigmoid/Tanh):

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{d_{in}+d_{out}}}, \sqrt{\frac{6}{d_{in}+d_{out}}}\right)$$

    where:
    - d_{in} is the number of input units to the layer
    - d_{out} is the number of output units from the layer
  - **He Initialization** (Standard for ReLU):

$$W \sim \mathcal{N}\left(0, \frac{2}{d_{in}}\right)$$

  - *Purpose:* Helps prevent vanishing or exploding gradients.
- **Biases** $b_1 = 0, b_2 = 0$ (typically initialized to zero)

Mapping Dimensions to Our MLP

We apply these formulas using our model's specific dimensions (d inputs, q hidden units):

Layer	**Weight Matrix**	**Shape**	$\mathbf{d_{in}}$	$\mathbf{d_{out}}$
Input $\rightarrow$ Hidden	W_1	$q \times d$	d	q
Hidden $\rightarrow$ Output	W_2	$1 \times q$	q	1

Specific Initializations:

- **For W_1 $(q \times d)$:**
  - Xavier term: $d_{in} + d_{out} = d + q$
  - He term: $d_{in} = d$
- **For W_2 $(1 \times q)$:**
  - Xavier term: $d_{in} + d_{out} = q + 1$
  - He term: $d_{in} = q$

Batch vs. Stochastic Gradient Descent

How much data do we use to calculate the gradient ∇L_θ before taking a step?

1. Batch Gradient Descent (Standard GD)
   - Uses the entire data set (n observations) to compute the gradient.
   - Problem: Very slow and computationally expensive if n is large (e.g., millions of images).
2. Stochastic Gradient Descent (SGD) with Mini-batches
   - Instead of the whole data set, we randomly sample a small subset of size s (e.g., $s = 32$ or 64).
   - This subset is called a **Mini-batch**.
   - We compute the gradient and update weights based *only* on this batch.
   - Benefit: Much faster updates and adds "noise" that can help escape bad local minima.

Terminology: What is an Epoch?

Now that we are processing data in batches, we need precise terminology for tracking progress.

Definition: An **Epoch** represents one complete pass through the entire training dataset.

During one epoch:

- The model effectively "sees" every training example exactly once.
- If we have n observations and batch size s, we must process $\frac{m}{s}$ batches to complete 1 epoch.

Analogy:

- **Data set:** A deck of 10000 observations.
- **Mini-batch:** Selecting 100 observations at a time.
- **Iteration:** Calculate parameter updates (through gradient descent algorithm) .
- **Epoch:** Once the algorithm has used at all 10K observations (13 iterations), it has completed one epoch.

Relationship: Epochs vs. Algorithm Iterations

- **Iteration (or Step):** One single update of the model parameters ($\theta \leftarrow \theta - \eta \nabla_\theta L$).
- **Epoch:** A cycle of iterations that covers the whole dataset.

The Mathematical Relationship: Let n be the training data set size and s be the mini-batch size. The number of batches (steps) per epoch is $r = \frac{n}{s}$.

Example Scenario

If we train for **100 epochs**:

- The outer loop runs 100 times.
- The inner loop (parameter updates) runs $100 \times r$ times.
- **Total Iterations** $= 100 \times \frac{m}{s}$.

Note: In Deep Learning, we typically fix the number of epochs rather than waiting for a specific tolerance ϵ.

Model Training, Tuning and Evaluating

We can not evaluate a model on the same data used to train it (this would be like memorizing the answer key). We split our data into three disjoint sets:

- 1. Training Set ($\approx 60 - 70\%$ of data)
  - Purpose: Used to compute gradients and update parameters (W, b).
  - *Size:* m examples.
- 2. Validation Set ($\approx 15 - 20\%$ of data)
  - Purpose: Used to tune hyperparameters (settings chosen *before* learning).
  - *In our MLP:* We use this set to choose the Hidden Dimension (q) and the batch size (a key parameter in the optimization algorithm).
  - It acts as a proxy for test performance during training.
- 3. Test Set ($\approx 15 - 20\%$ of data)
  - Purpose: Used **only once** at the very end to estimate real-world generalization.
  - *Golden Rule:* Never train on the test set!

Binary Classification Evaluation Basics - I

We evaluate performance on a test data set of size N_{test}.

- True labels: $y_i \in \{0, 1\}, \; i = 1, \cdots, N_{\text{test}}$
- Predicted labels: $\hat{\hat{y}}_i \in \{0, 1\}, \; i = 1, \cdots, N_{\text{test}}$

Both Logistic Regression and MLPs output a probability:

$$\hat{y}_i = \mathbb{P}(y_i = 1 | x_i)$$

To obtain the discrete prediction $\hat{\hat{y}}_i$, we apply a threshold $t \in (0, 1)$:

$$\hat{\hat{y}}_i = \begin{cases} 1 & \text{if } \hat{y}_i > t \\ 0 & \text{otherwise} \end{cases}$$

The standard default threshold is $t = 0.5$.

Binary Classification Evaluation Basics - II

The Confusion Matrix

		Predicted	
		Positive ($\hat{\tilde{y}}_i = 1$)	Negative ($\hat{\tilde{y}}_i = 0$)
Actual	Positive ($y_i = 1$)	True Positive (TP)	False Negative (FN)
	Negative ($y_i = 0$)	False Positive (FP)	True Negative (TN)

Key performance metrics are derived directly from these four quadrants.

Evaluation Metrics - I

Accuracy

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Interpretation: The proportion of total predictions that are correct.

Limitation: Accuracy is misleading for imbalanced classes.

- Example: If 95% of observations are negative, a "dumb" model that always predicts 0 will achieve 95% accuracy but fails to detect any positive cases.

Metrics - II

Precision and Recall

- **Precision (Positive Predictive Value):**

$$\text{Precision} = \frac{TP}{TP + FP}$$

Interpretation: Of all instances predicted as positive, how many were actually positive?

- **Recall (Sensitivity or TPR):**

$$\text{Recall} = \frac{TP}{TP + FN}$$

Interpretation: Of all actual positive instances, how many did the model correctly identify?

Metrics - III

F1 Score

$$\text{F1 Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

- The harmonic mean of Precision and Recall.
- **Use Case:** Preferred when there is a class imbalance and we need to balance the trade-off between precision and recall.

Metrics - IV

False Positive Rate (FPR) and True Positive Rate (TPR)

- **True Positive Rate (TPR) / Recall:**

$$TPR = \frac{TP}{TP + FN}$$

- **False Positive Rate (FPR):**

$$FPR = \frac{FP}{FP + TN}$$

Interpretation: The proportion of negative instances that are incorrectly classified as positive.

Metrics - V

Specificity

$$\text{Specificity} = \frac{TN}{TN + FP} = 1 - \text{FPR}$$

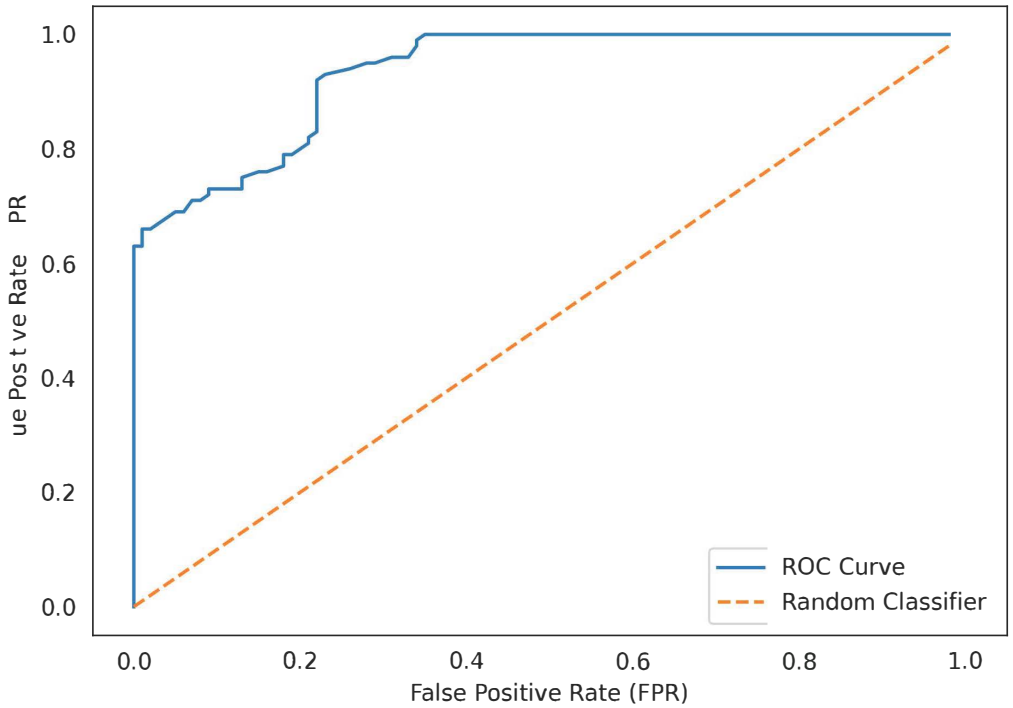
- Also known as the True Negative Rate (TNR).
- **Interpretation:** Measures the model's ability to correctly identify negative instances.

Which metric to use and when?

- **Imbalanced Classes:** Avoid Accuracy. Use Precision, Recall, or F1 Score.
- **High Cost of False Negatives (e.g., Medical Tests):** Prioritize High Recall (we don't want to miss a disease).
- **High Cost of False Positives (e.g., Spam Detection):** Prioritize High Precision (we don't want to delete legitimate emails).

Receiver Operating Characteristic (ROC) Curve

- Plots **TPR** (y-axis) vs. **FPR** (x-axis) at various classification thresholds $t \in (0, 1)$.
- Visualizes the trade-off between sensitivity and specificity.
- A perfect classifier reaches the top-left corner ($TPR = 1, FPR = 0$).



AUC - Area Under the ROC Curve

- AUC measures the entire 2-dimensional area underneath the ROC curve.
- Range: 0.5 (Random Guessing) to 1.0 (Perfect Classifier).
- Interpretation: The probability that a randomly chosen positive instance is ranked higher than a randomly chosen negative instance.

Intuitive Interpretation of AUC

The "Ranking" Experiment:

1. Take one random sample from the positive class (x^+).
2. Take one random sample from the negative class (x^-).
3. Use the model to assign scores $s(x)$ to both.

Question: What is the probability that $s(x^+) > s(x^-)$?

Answer: This probability is exactly the AUC!

$$\text{AUC} = \mathbb{P}(s(x^+) > s(x^-))$$

Note: This assumes the classifier outputs a real-valued score (e.g., probability).

Experimental Setup: Binary Classification - I

- **Sample Size:** $N = 10,000$, split into:
  - Training: $N_{\text{train}} = 6,000$
  - Validation: $N_{\text{val}} = 2,000$
  - Test: $N_{\text{test}} = 2,000$
- **Features:** $p = 100$ covariates, with average correlation > 0.4.
- **Data Generation:** y_i generated via logistic regression model with $c\%$ label noise.
- **Models:**
  - Logistic Regression: Uses training data to estimate β, test data to evaluate.
  - MLP: Uses training data for gradients, validation data to tune hyperparameters (mini-batch size, hidden dimension), and test data for final evaluation.

Experimental Setup: Binary Classification - II

Generating $y_i \in \{0, 1\}$:
The true underlying model is:

$$\mathbb{P}(y_i = 1 \mid x_i) = \sigma(x_i^\top \beta) = \frac{1}{1 + \exp(-x_i^\top \beta)}, \tag{2}$$

where $x_i \in \mathbb{R}^p$ and $\beta \in \mathbb{R}^p$.

Label Noise: To simulate noise, we flip labels for a random subset. For noise level $c \in [0, 1]$:

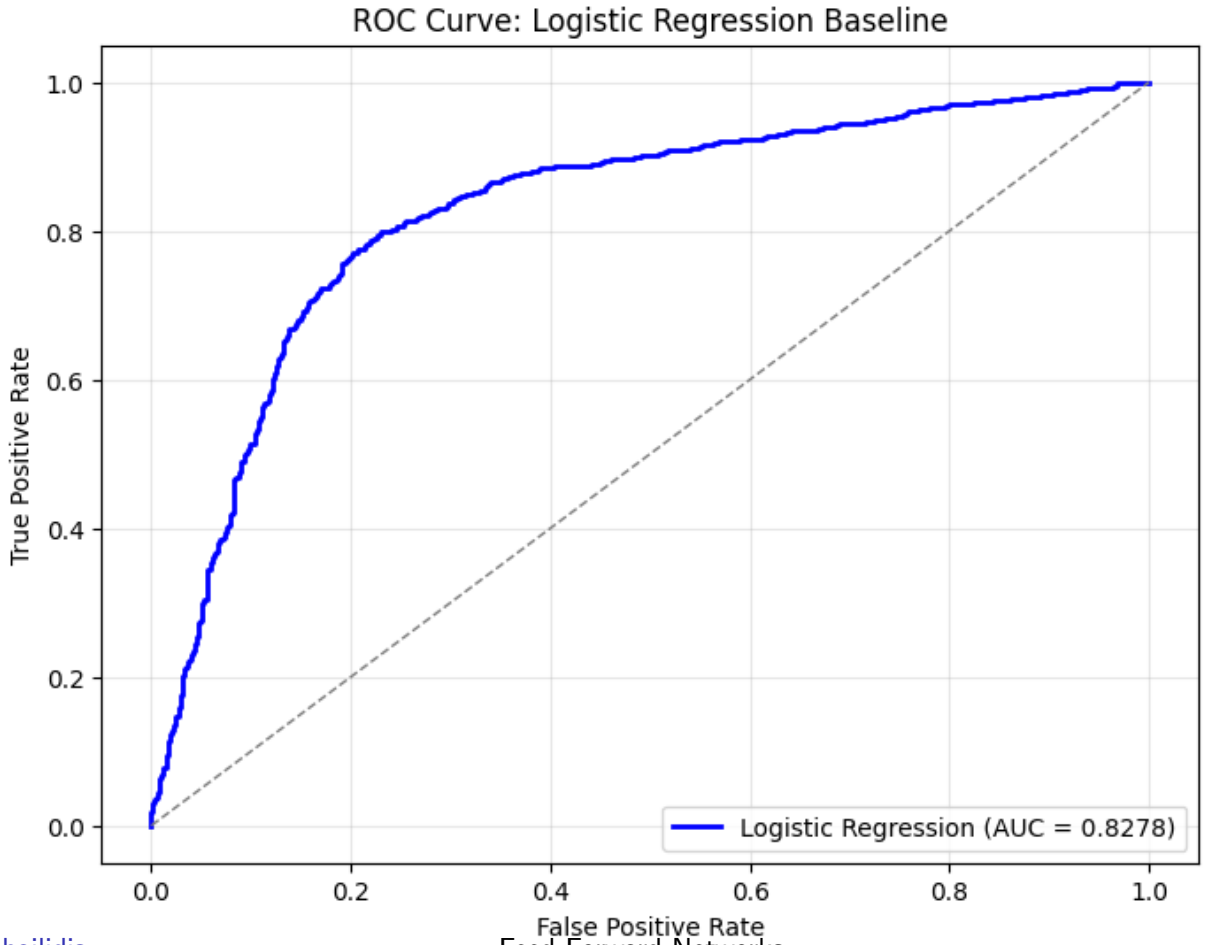
1. Randomly select $\lfloor cN \rfloor$ indices $\mathcal{I}_{\text{flip}} \subset \{1, \ldots, N\}$.
2. Flip labels: $y_i \leftarrow 1 - y_i$ for all $i \in \mathcal{I}_{\text{flip}}$.

Higher c implies a harder classification task.

The Gold Standard: Logistic Regression

Since the data is generated by a logistic model, Logistic Regression provides the theoretical upper bound on performance.

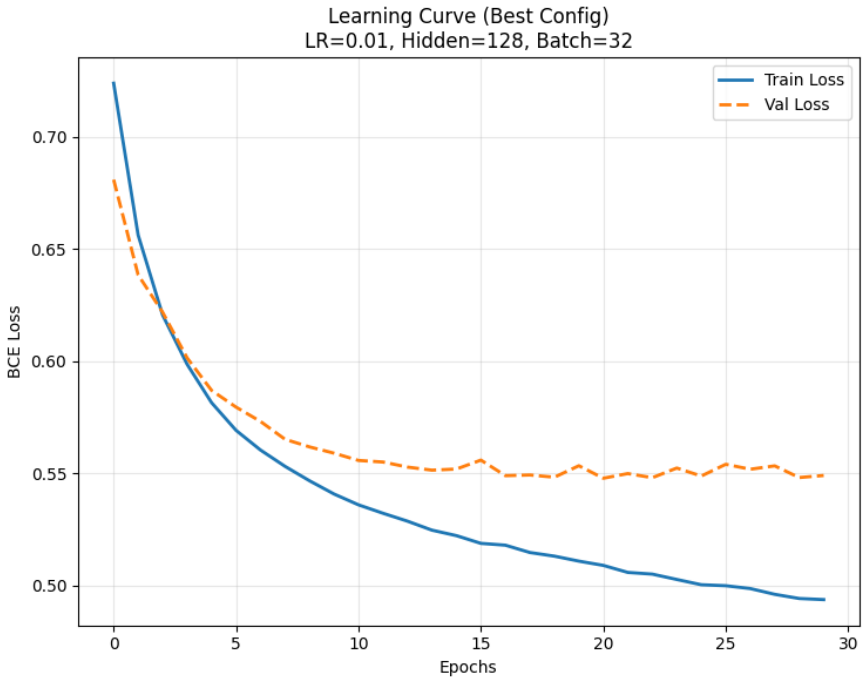
Scenario: Label noise proportion $c = 0.15$.

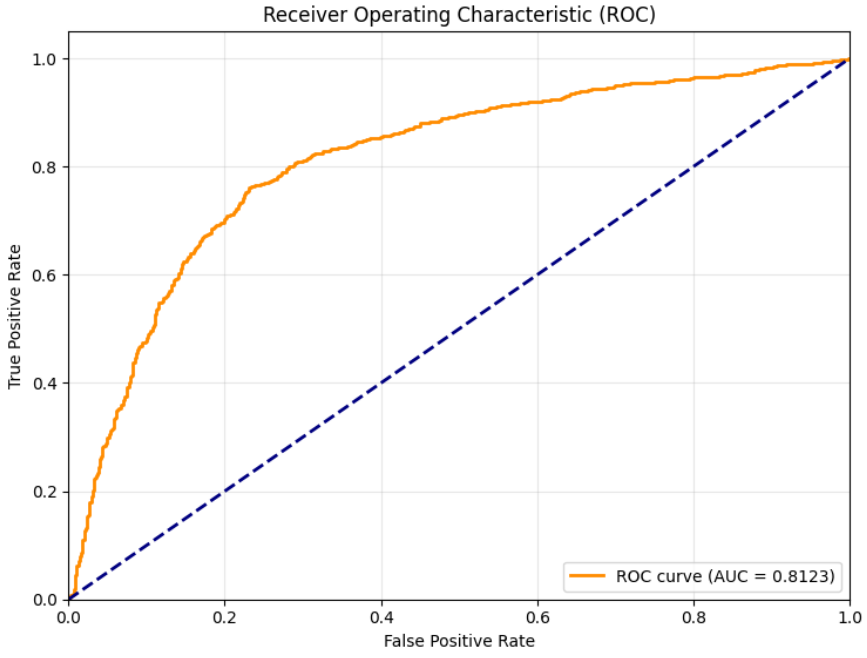


Hyperparameters for MLP

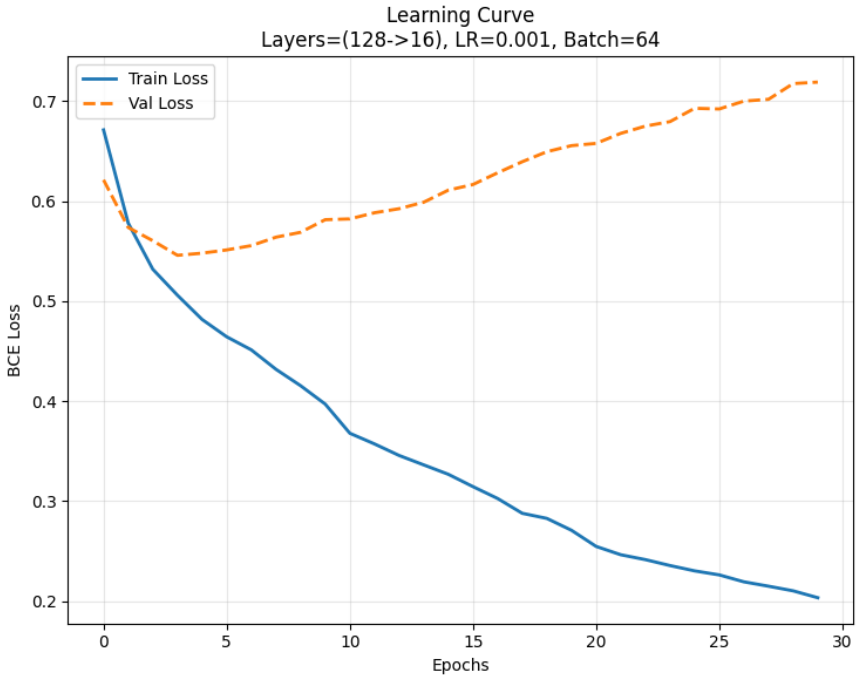
- **Epochs:** 50
- **Learning rate**
- **Hidden Dimension (q)**
- **Mini-batch Size (s)**

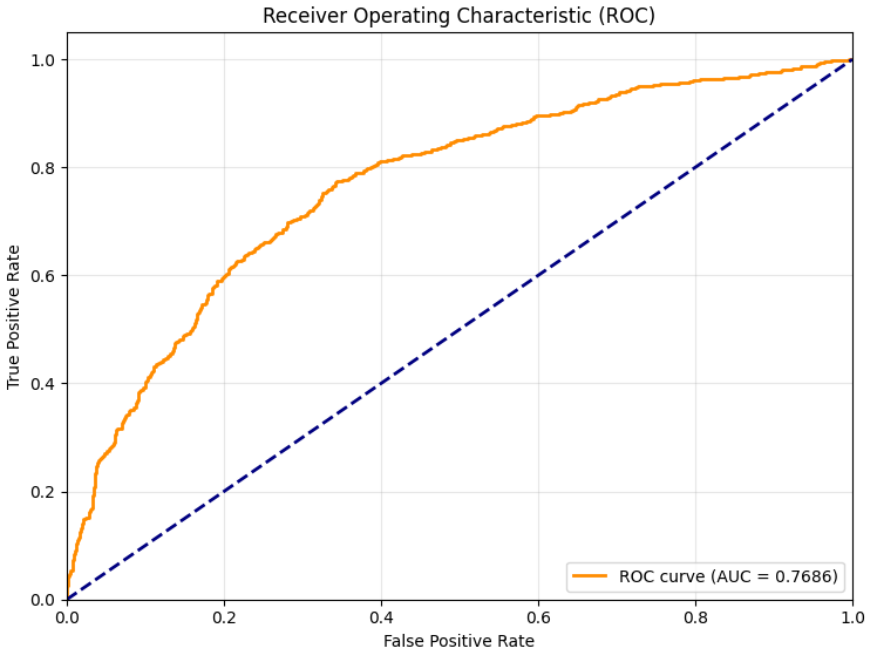
MLP Performance





Deepen the architecture, 2-layer MLP





Regularization Strategies for Deep Neural Networks

1. Dropout
2. Weight Decay
3. Early Stopping

1. Dropout: Robustness via Noise Injection

Key Intuition:

- A well trained model should be robust to small perturbations in the input features.
- Dropout extends this by injecting noise internally into hidden layers.
- It prevents co-adaptation: Neurons cannot rely on the presence of specific other neurons to correct their mistakes. They must learn features that are useful in many different contexts.

The Mechanism: Given a hidden activation h, dropout replaces it with h':

$$h' = \begin{cases} 0 & \text{with probability } p \\ \frac{h}{1-p} & \text{with probability } 1-p \end{cases} \tag{3}$$

- Inverted Dropout: We divide by $(1-p)$ during training to keep the *expected value* of the activation unchanged: $E[h'] = h$.
- This ensures that during testing (when dropout is off), the layer outputs do not shift in magnitude.

2. Implementation & Theoretical Links

Training Phase (Stochastic):

- For every mini-batch, a different random subset of neurons is deactivated.
- Effectively trains an ensemble of 2^N thinned networks that share weights.

Testing/Inference Phase (Deterministic):

- Dropout is **disabled**. All neurons are active.
- Because we applied scaling ($\frac{1}{1-p}$) during training, no weight adjustment is needed here.

Theoretical Connection: Weight Decay

Research (Bishop, 1995) showed that training with input noise is mathematically equivalent to Tikhonov Regularization (Weight Decay). Dropout generalizes this by adding noise to internal representations, implicitly penalizing complexity.

3. Beyond Dropout: Other Strategies

1. Weight Decay (ℓ_2 Regularization):
   - Adds a penalty term to the loss function: $Ł_{reg} = L_{data} + \frac{\lambda}{2}||\mathbf{w}||^2$.
   - Encourages weights to remain small/diffuse.
   - **Intuition:** Small weights mean the function is smoother and less sensitive to changes in input.
2. Early Stopping:
   - Stop training when Validation Error starts to rise, even if Training Error is still falling.
   - Acts as a proxy for controlling model complexity (limiting the "time" the model has to fit noise).
3. Data Augmentation:
   - Artificially expanding the dataset (e.g., for image data add rotations, flips, noise) to force invariance.
   - Related to Dropout: adding noise to the *input* vs. *activations*.